

一、概论

1.宏内核：

整个系统分为内核与应用两层

内核：运行在特权级，集中控制所有计算资源

应用：运行在非特权级，受内核管理，使用内核服务

优缺点：

- 宏内核拥有丰富的沉淀和积累
 - 拥有巨大的统一的社区和生态
 - 针对不同场景优化了 30 年
- 宏内核的结构性缺陷
 - 安全性与可靠性问题：模块之间没有很强的隔离机制
 - 实时性支持：系统太复杂导致无法做最坏情况时延分析
 - 系统过于庞大而阻碍了创新：Linux 代码行数已经过 2800 万

难以满足：

- 向上向下的扩展
 - 很难去剪裁/扩展一个宏内核系统支持从 KB 级别到 TB 级别的场景
- 硬件异构性
 - 很难长期支持一些定制化的方式去解决一些特定问题
- 功能安全
 - 一个广泛共识：Linux 无法通过汽车安全完整性认证 (ASIL-D)
- 信息安全
 - 单点错误会导致整个系统出错，而现在有数百个安全问题 (CVE)
- 确定性时延
 - Linux 花费 10+ 年合并实时补丁，目前依然不确定是否能支持确定性时延

2.微内核：

- 设计原则：最小化内核功能
 - 将操作系统功能移到用户态，称为“服务” (Server)
 - 在用户模块之间，使用消息传递机制通信
- 优点
 - 易于扩展：直接添加一个用户进程即可为操作系统增加服务
 - 易于移植：大部分模块与底层硬件无关
 - 更加可靠：在内核模式运行的代码量大减少
 - 更加安全：即使存在漏洞，服务与服务之间存在进程粒度隔离
 - 更加健壮：单个模块出现问题不会影响到系统整体
- 缺点

- 性能较差：内核中的模块交互由函数调用变成了进程间通信
- 生态欠缺：尚未形成像 Linux 一样具有广泛开发者的社区
- 重用问题：重用宏内核操作系统提供兼容性，带来新问题

混合内核架构：

- 宏内核与微内核的结合
 - 将需要性能的模块重新放回内核态

3.外核架构：

- Exokernel 不提供硬件抽象
 - “只要内核提供抽象，就不能实现性能最大化”
 - 只有应用才知道最适合的抽象 (end-to-end 原则)
- Exokernel 不管理资源，只管理应用
 - 负责将计算资源与应用的绑定，以及资源的回收
 - 保证多个应用之间的隔离

库 OS (LibOS)

- 策略与机制分离：将对硬件的抽象以库的形式提供
- 高度定制化：不同应用可使用不同的 LibOS，或完全自定义
- 更高性能：LibOS 与应用其他代码之间通过函数调用直接交互

Unikernel (单内核)

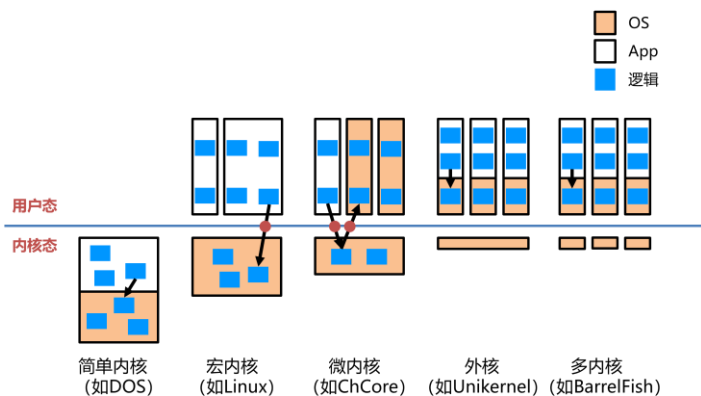
- 虚拟化环境下的 LibOS
 - 每个虚拟机只使用内核态
 - 内核态中只运行一个应用+LibOS
 - 通过虚拟化层实现不同实例间的隔离
- 适合容器等新的应用场景
 - 每个容器就是一个虚拟机
 - 每个容器运行定制的 LibOS 以提高性能

Exokernel 架构的优缺点分析

- 优点
 - OS 无抽象，能在理论上提供最优性能
 - 应用对计算有更精确的实时等控制
 - LibOS 在用户态更易调试，调试周期更短
- 缺点
 - 对计算资源的利用效率主要由应用决定
 - 定制化过多，导致维护难度增加

4.多内核/复内核 (Multikernel)

- 背景：多核与异构
 - OS 内部维护很多共享状态
 - Cache 一致性的保证越来越难
 - 可扩展性非常差，核数增多，性能不升反降



- GPU 等设备越来越多
 - 设备本身越来越智能——设备有自己的 CPU
 - 通过 PCIe 连接，主 CPU 与设备 CPU 之间通信非常慢
 - 通过系统总线连接，异构 SoC (System on Chip)
- Multikernel 的思路
 - 默认的状态是划分而不是共享
 - 维持多份状态的 copy 而不是共享一份状态
 - 显式的核间通信机制
- Multikernel 的设计
 - 在每个 core 上运行一个小内核
 - 包括 CPU、GPU 等
 - OS 整体是一个分布式系统
 - 应用程序依然运行在 OS 之上

二、底层基础

- ISA (Instruction Set Architecture)
 - CPU 向软件（应用程序和操作系统）提供的接口
 - 理解软件在 CPU 上的运行（操作系统设计、程序调试、内存安全等）
 - 操作系统中包含体系结构相关的汇编代码
 - 操作系统启动代码（例如栈未设置）
 - 部分操作 C 语言无法表达（例如获取系统状态、刷新 TLB）
 - 有场景汇编代码比 C 代码高效很多（例如 memcpy）
- ARMv8: 31 个 64 位通用寄存器：处理器内部的高速存储单元。64 位 X，其中小 32 位 W。
PC 寄存器，PSTATE，ZR 寄存器，SP 寄存器。

指令部分

- mov dst, src 可为 32/64 位。src 还可为立即数如 #1。高 32 位清 0
- add/sub/mul/div Rd,Rn,Op2

- neg Rd,Rn 相反数，控制符号
- lsl/lsr Rd,Rn,Op2 位左右移 补 0
- asr/ror 算数右移（不改变符号/循环）
- eor/orr/and Rd,Rn,Op2
- mvn Rd,Rn 位取反
- add x2, x2, x2, lsl #1 合法，跳过中间变量**
- 对操作数进行位扩展**
- 例子：add x19, x19, w0, sxtw
- 无符号扩展: uxtb, uxth, uxtw
- 符号扩展: sxtb, sxth, sxtw
- ldr R,addr R ← mem[addr : addr+R_s] (64 位寻址)
- str R,addr mem[addr : addr+R_s] ← R
- ldp R1, R2, addr R1,R2 ← mem[addr : addr+R1_s+R2_s]
- stp R1, R2, addr mem[addr : addr+R1_s+R2_s] ← R1,R2
- 寻址模式是表示内存地址的表达式**
 - 基地址模式（索引寻址）
 - [r_b]
 - 基地址加偏移量模式（偏移量寻址）
 - [r_b, offset]
 - 前索引寻址（寻址操作前更新基地址）
 - [r_b, offset]! r_b += Offset; 寻址 M[r_b]
 - 后索引寻址（寻址操作后更新基地址）
 - [r_b], offset 寻址 M[r_b]; r_b += Offset
- 条件码是一组标志位的统称**
 - 条件码由 PSTATE 寄存器维护
 - N (Negative)、Z (Zero)、C (Carry)、V (Overflow)
- 条件码保留之前相关指令的执行状态，这种指令包括：**
 - 带有 s 后缀的算术或逻辑运算指令（如 subs、adds）
 - 比较指令**
 - cmp: 操作数之差；例如 cmp x0, x1
 - cmn: 操作数之和；例如 cmn x0, x1
 - tst : 操作数相与；例如 tst x0, x1

函数部分

- 指令
 - bl label (直接调用，调用函数)
 - blr Rn (间接调用，调用函数指针)
- 功能
 - 将返回地址存储在链接寄存器 LR (x30)

寄存器的别名)

- 跳转到被调用者的入口地址
- 指令
 - **ret** (不区分直接调用与间接调用)
- 功能
 - 跳转到返回地址 (链接寄存器 LR, Link Register)
- 栈帧: 每个函数在运行期间使用的一段内存
 - 生命周期: 从被调用到返回前
 - 作用: 存放其局部状态
 - 存放返回地址
 - 存放上一个栈帧的位置
 - 存放局部变量
- CPU 中的另一个特殊寄存器 SP (独立)
 - 指向栈顶 (低地址)

帧指针 FP: x29 寄存器

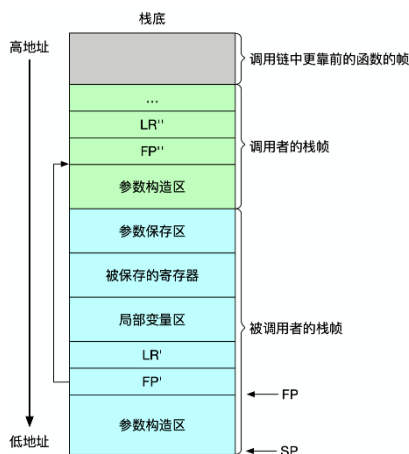
- 栈帧回溯
 - 栈帧大小不一
 - 如何找到上一个栈帧 (如调试)
 - 保存 x29 (上一个栈帧的 SP)
 - 将当前 SP 写入 x29 (让 callee 能保存)

似乎是修改和保存的中间量, 在新 SP 下保存旧 SP

- 调用者使用 x0 ~ x7 寄存器传递前 8 个参数
- 被调用者使用 x0 寄存器传递返回值
- 调用者压到栈上的数据
 - 第 8 个之后的参数
 - 按声明顺序从右到左
 - 所有数据对齐到 8 字节
- 被调用者通过 SP+偏移量访问
- 约定: x9-x15 调用者保存; x19-x28 被调用者保存

按需保存, 调用完恢复

函数局部变量存放在函数栈帧中



系统部分

- 抽象进程状态信息 (PSTATE)
 - 条件码 (Condition flags)

- NZCV

- 执行状态 (Execution state controls)
 - **CurrentEL: CPU 当前特权级别**
- 异常掩码 (Exception mask bits)
 - DAIF
- 访问控制 (Access control bits)
 - 例如 PAN (Privileged Access Never)

• 用户态 (User-mode)

- 只能使用用户 ISA

• 内核态 (Kernel-mode)

- 可以同时使用系统 ISA 和用户 ISA

• 操作系统往往同时包含内核态与用户态的代码

- 如: Unix 包含内核态的 kernel 与 用户态的 shell

寄存器	ELO	EL1	描述
通用寄存器	X0 ~ X30	✓	✓
特殊寄存器	PC	✓	✓
	SP_ELO	✓	✓
	SP_EL1	✓	✓
	PSTATE	✓	✓
系统寄存器	ELR_EL1	✓	✓
	SPSR_EL1	✓	✓
	VBAR_EL1	✓	✓
	ESR_EL1	✓	✓

• 同步异常

- 执行当前指令触发异常
 - 第一类: 用户程序主动发起: **svc 指令** (OS 利用 eret 指令返回)
 - 第二类: 非主动, 例如用户程序意外访问空指针: 普通 ldr 指令 (OS“杀死”出错程序)

• 异步异常

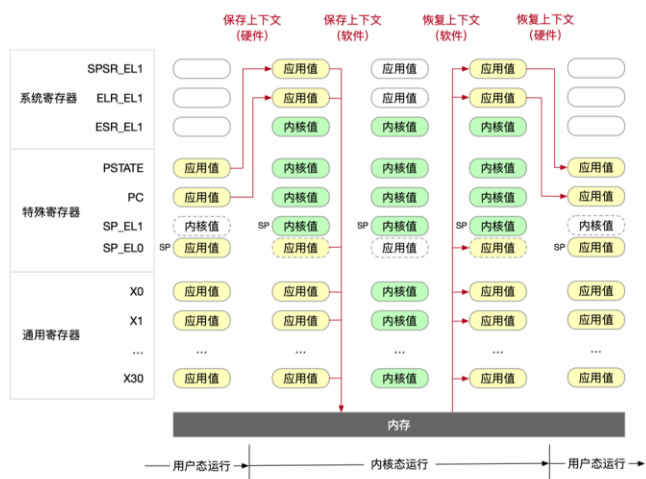
- CPU 收到中断信号
 - 从外设发来的中断, 例如屏幕点击、鼠标、收到网络包
 - CPU 时钟中断, 例如定时器超时

异常处理函数

- 属于操作系统的一部分
 - 运行在内核态的代码
- 异常处理函数完成异常处理后, 将通过下述操作之一转移控制权:
 - 回到发生异常时正在执行的指令 (如外部中断)
 - 回到发生异常时的下一条指令 (空指针访问等)
 - 不返回/切换到其它进程执行 (kill)

异常向量表：CPU 找到异常处理函数

- 操作系统内核预先在一张表中设置不同异常的处理函数
 - 基地址存储在 VBAR_EL1 寄存器中
 - 系统寄存器
- CPU 在异常发生时自动跳转到相应处理函数
 - 同步异常：主动下陷 svc、指令执行出错
 - 异步异常：中断（IRQ、FIQ）、SError



处理器（硬件）在切换过程中的任务

- 将发生异常事件的指令地址保存在 ELR_EL1 中
- 将异常事件的原因保存在 ESR_EL1

例如，是执行 svc 指令导致的，还是访问地址出错导致的

- 将处理器的当前状态（即 PSTATE）保存在 SPSR_EL1
- 栈寄存器不再使用 SP_ELO（用户态栈寄存器），开始使用 SP_EL1
内核态栈寄存器，需要由操作系统提前设置
- 修改 PSTATE 寄存器中的特权级标志位，设置为内核态
- 找到异常处理函数的入口地址，并将该地址写入 PC，开始运行操作系统
根据 VBAR_EL1 寄存器保存的异常向量表基地址，以及发生异常事件的类型确定

eret：从内核态返回到用户态

- 将 SPSR_EL1 中的处理器状态写入 PSTATE 中
 - 处理器状态也从 EL1 切换到 ELO
- 栈寄存器不再使用 SP_EL1，开始使用 SP_ELO
 - 注意：SP_EL1 的值并没有改变
 - 下一次下陷时，操作系统依然会使用这个内核栈
- 将 ELR_EL1 中的地址写入 PC，并执行应用程序代码

操作系统（软件）在切换过程中的任务

- 主要任务：将属于应用程序的 CPU 状态保存到内存中
 - 用于之后恢复应用程序继续运行
- 应用程序需要保存的运行状态称为**处理器上下文**
 - 处理器上下文（Processor Context）：应用程序在完成切换后恢复执行所需的最小处理器状态集合
 - 处理器上下文中的寄存器具体包括：
 - 通用寄存器 X0-X30
 - 特殊寄存器，主要包括 PC、SP 和 PSTATE
 - 系统寄存器，包括页表基地址寄存器

系统调用的参数传递和返回值（软件约定）

- 最多允许 8 个参数
 - x0-x7 寄存器
 - x8 用于存放系统调用编号
- 返回值存放于 x0 寄存器中

vDSO：直接加载代码和值（只读）避免系统调用

- 内核定义
 - 在编译时作为内核的一部分
- 用户态运行
 - 将 gettimeofday（例子）的代码加载到一块与应用共享的内存页
 - 这个页称为：vDSO
 - Virtual Dynamic Shared Object
 - Time 的值同样映射到用户态空间（只读）
 - 只有在内核态才能更新这个值

Flex-SC

- 一种新的 syscall 机制
 - 引入 system call page，由 user & kernel 共享
 - 应用程序可以将系统调用的请求 push 到 system call page
 - 内核会从 system call page poll system call 请求
 - Exception-less syscall
 - 将系统调用的调用和执行解耦，可分布到不同的 CPU 核

Privbox：不想看

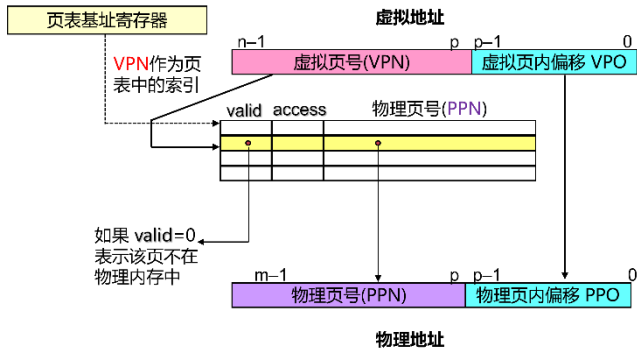
三、内存管理

Memory Management Unit（MMU）

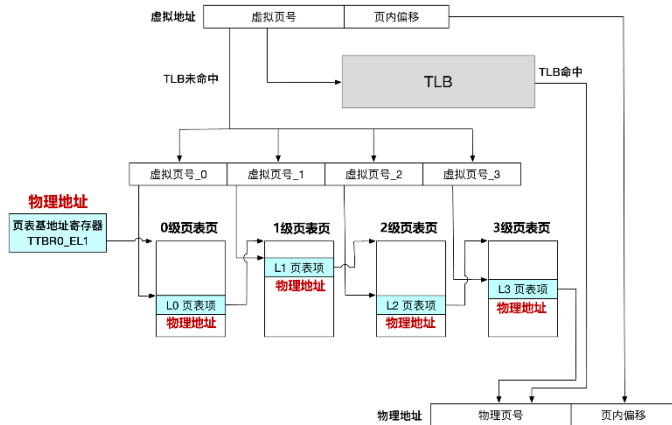
- 如果使用单级页表结构，那么页表本身有多大呢？
 - 假设 32 位的地址空间，页大小为 4K，页表项为 4 字节
 - 假设 64 位的地址空间，页大小为 4K，

页表项为 8 字节

- 对于 32 位地址空间
 - $2^{32} / 4K * 4 = 4MB$
- 对于 64 位地址空间
 - $2^{64} / 4K * 8 = 33,554,432 \text{ GB}$



- 多级页表能有效压缩页表的大小
- 原因：允许页表中出现空洞
 - 若某个页表页中的某个页表项为空，那么其对应的下一级页表页便无需存在
 - 应用程序的虚拟地址空间大部分都未分配



- [63: 48] 16-bit
 - 必须全是 0 或者全是 1（软件用法：一般应用程序地址是 0，内核地址是 1）
 - 也意味着虚拟地址空间大小最大是 2^{48} Byte，即 256TB
- [47: 39] 9-bit: 0 级页表索引
- [38: 30] 9-bit: 1 级页表索引
- [29: 21] 9-bit: 2 级页表索引
- [20: 12] 9-bit: 3 级页表索引
- [11: 0] 12-bit: 页内偏移

假设进程只用到 0, 1, $2^{36} - 1$ 三个虚拟页

- 单级页表, 共需 2^{36} 个页表项, 共占用 $2^{39}B$
- 4 级页表, 仅需要一个 0 级页表, 2 个一级页表, 2 个二级页表, 2 个三级页表, 共七个页表, 每个页表页占用 (4KB), 共占用 28KB

若在页表中填写虚拟地址 0-16M 的映射, 则页表至少需要占用多少空间? 虚拟地址对应的 0、1、2、3 级页表项索引分别是 1、2、3、4。

44KB。因为需要 1 个 0 级页表页, 1 个 1 级页表页, 1 个 2 级页表, 8 个 3 级页表页, 合计 11 个页表页。

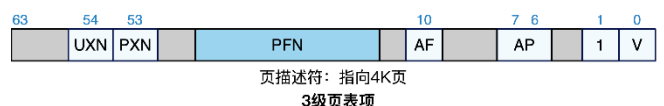
- AARCH64
 - 两个页表基地址寄存器: TTBR0_EL1 & TTBR1_EL1
 - MMU 根据虚拟地址第 63 位选择使用哪一个
 - 以 Linux 为例
 - 应用程序 (地址首位为 0): 使用 TTBR0_EL1
 - 操作系统 (地址首位为 1): 使用 TTBR1_EL1
- X86_64
 - 一个寄存器: CR3 (Control Register 3)
- 机器上电会先进入物理寻址模式
 - 系统软件需配置控制寄存器使能页表从而进入虚拟寻址模式
- AARCH64
 - SCTLR_EL1, System Control Register, EL1
 - 第 0 位 (M 位) 置 1, 即在 EL0 和 EL1 权限级使能页表
- X86_64
 - CR4, 第 31 位 (PG 位) 置 1, 即使能页表

页表页

- 每级页表有若干离散的页表页
 - 每个页表页占用一个物理页
- 第 0 级 (顶层) 页表有且仅有一个页表页
 - 页表基地址寄存器存储的就是该页的物理地址
- 每个页表页中有 512 个页表项
 - 每项为 8 个字节 (64 位), 用于存储物理地址和权限

页表项中的属性位

- V (Valid):** 当访问时 $V=0$, 则触发缺页异常
- UXN (Unprivileged eXecute Never): 用户态不可执行 (例如栈)
- PXN (Privileged eXecute Never): 内核态不可执行 (防止内核执行用户代码)
- AF (Access Flag): 当访问时, MMU 标记该位
- AP (Access Permissions)
- DBM (Dirty Bit): 51 位, ARMv8.1-TTBM 特性支持



有效的 3 级页表项: 第 1 位必须是 1

前 3 级只有 PFN 和最后两位。第一位如为 1 则块

- 多级页表的设计是典型的用时间换空间的设计
 - 优点：压缩页表大小
 - 缺点：增加了访存次数（逐级查询）
 - 1 次内存访问，变成了 5 次内存访问

TLB

- TLB 位于 CPU 内部，是页表的缓存
 - Translation Lookaside Buffer
 - 缓存了虚拟页号到物理页号的映射关系
 - 有限数目的 TLB 缓存项
- 在地址翻译过程中，MMU 首先查询 TLB
 - TLB 命中，则不再查询页表（fast path）
 - TLB 未命中，再查询页表

地址翻译

- 虚拟地址（VA）构成
 - VPO（Virtual page offset），虚拟页偏移
 - VPN（Virtual page number），虚拟页号
 - TLBI（TLB index）
 - TLBT（TLB tag）
- 物理地址（PA）构成
 - PPO（Physical page offset），物理页偏移
 - PPN（Physical page number），物理页号
- TLB 使用虚拟地址索引
 - 当 OS 切换进程页表时需要全部刷新
- AARCH64 上内核和应用程序使用不同的页表
 - 分别存在 TTBR0_EL1 和 TTBR1_EL1
 - 使用的虚拟地址范围不同
 - 系统调用过程不用切换

如何降低 TLB 刷新导致的开销

- 硬件特性 ASID（AARCH64）：Address Space ID
 - OS 为不同进程分配 8 位或 16 位 ASID
 - ASID 的位数由 TCR_EL1 的第 36 位（AS 位）决定
 - OS 负责将 ASID 填写在 TTBR0_EL1 的高 8 位或高 16 位
 - TLB 的每一项也会缓存 ASID
 - 地址翻译时，硬件会将 TLB 项的 ASID 与 TTBR0_EL1 的 ASID 对比
 - 若不匹配，则 TLB miss
- 使用了 ASID 之后
 - 切换页表（即切换进程）后，不再需要刷新 TLB，提高性能
 - 修改页表映射后，仍需刷新 TLB

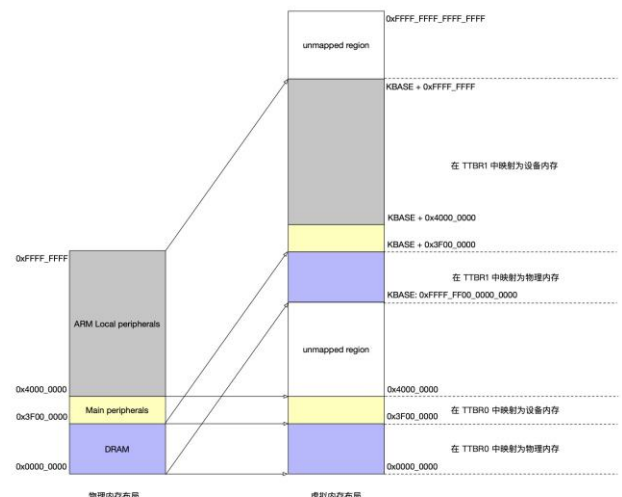
- OS 修改页表后，需要刷新其它核的 TLB 吗？
 - 需要，因为一个进程可能在多个核上运行。那成本挺高啊
- OS 如何知道需要刷新哪些核的 TLB？
 - 操作系统知道进程调度信息
- OS 如何刷新其他核的 TLB？
 - x86_64: 发送 IPI 中断某个核，通知它主动刷新
 - AARCH64: 可在 local CPU 上刷新其它核 TLB
 - 调用的 ARM 指令：TLBI ASIDE1IS
- 刷新 TLB 的相关指令
 - 清空全部 TLBI VMALLEL1IS
 - 清空指定 ASID TLBI ASIDE1IS
 - 清空指定虚拟地址 TLBI VAE1IS

不是整个都刷新

虚拟内存机制的优势

- 高效使用物理内存
 - 使用 DRAM 作为虚拟地址空间的缓存
- 简化内存管理
 - 每个进程看到的是统一的线性地址空间
- 更强的隔离与更细的权限控制
 - 一个进程不能访问属于其他进程的内存
 - 用户程序不能够访问特权更高的内核信息
 - 不同内存页的读、写、执行权限可以不同
- 不同进程互不干扰
 - 仿佛独占所有内存
- 绝大部分地址段均可用
 - 除了顶部的内核地址区域
- 不同的进程对相同的物理页拥有不同的权限
 - 通过页表项中的权限位来控制

四、启动



ARMv8 架构下内核启动的 3 个主要任务

- 设置 CPU 异常级别（特权级别）
- 设置页表并开启虚拟内存机制
- 设置异常向量表并打开中断具体过程
- 设置 CPU 异常级别为 EL1
- 设置页表并开启虚拟内存机制
 - TTBR0_EL1: 虚拟地址 = 物理地址
 - TTBR1_EL1: 虚拟地址 = 物理地址 + OFFSET
- 设置异常向量表
 - 每个异常向量表项跳转到对应的异常处理函数
 - 处理异常前保存进程上下文、返回进程前恢复其上下文

TTBR1_EL1 和 TTBR0_EL1 分别指向内核和应用页表

- 操作系统自己使用的页表
 - 在启动时填写
 - 映射全部物理内存
 - 虚拟地址 = 物理地址 + 固定偏移 直接映射, Direct Mapping

立即映射

- 创建进程时, OS 按照虚拟内存区域填写进程页表
 - 例如, 代码段和数据段
 - 具体步骤:
 - 步骤 -1: 分配物理页 (alloc_page)
 - 步骤-2: 把应用代码/数据从磁盘加载到物理页中
 - 步骤-3: 添加虚拟页到物理页的映射 (add_mapping)
 - 步骤-4: 未加载完毕, 回到步骤-1

操作系统在进程结构体中保存页表基地址

- 立即映射是一种操作系统可以选择的页表填写策略
 - 在初始化进程虚拟地址空间时, 直接在进程页表中添加各虚拟内存区域的映射
- 潜在弊端
 - 以关卡/副本类型游戏加载为例: 只玩 1 关, 加载 1000 关
 - 物理内存资源浪费
 - 非必要时延

延迟映射

- 解决立即映射弊端的直观想法
 - 操作系统按进程实际需要分配物理页和填写页表, 避免分配的物理页实际不被用到的情况
- 主要思路: 解耦虚拟内存分配与物理内存分配

- 先记录下为进程分配的虚拟内存区域
- 当进程实际访问某个虚拟页时, CPU 会触发缺页异常
- 操作系统在缺页异常处理函数中添加映射
- V (Valid): 当 MMU 查询页表项时, 若 V=0, 则触发缺页异常

虚拟地址空间

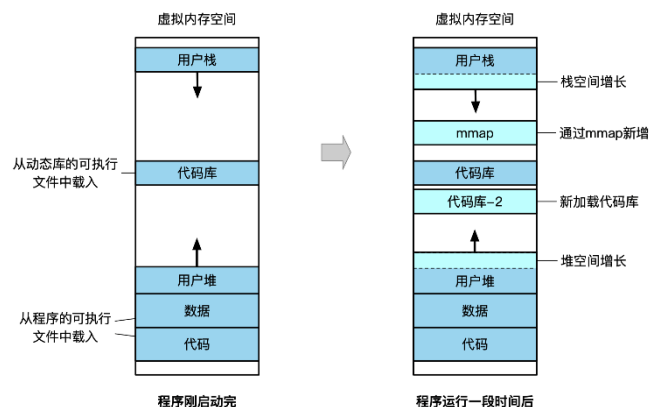
- 若干非连续的虚拟内存区域
 - 每个虚拟内存域中的虚拟地址都是进程可用的
 - 相同的访问权限
 - 例如: 代码、数据、堆、栈
 - 非法虚拟地址, 访问触发 CPU 异常
 - 操作系统会报 segfault

segfault (软件行为) vs. pagefault (硬件异常)

- 记录进程已分配的虚拟内存区域
 - 在 Linux 中对应 vm_area_struct (VMA) 结构体
 - 在 ChCore-Lab 中对应 vmregion (vmr) 结构体

添加 VMA

- 途径-1: OS 在创建进程时分配
 - 数据 (对应 ELF 段)
 - 代码 (对应 ELF 段)
 - 栈 (初始无内容)



- 途径-2: 进程运行时添加
 - 堆、栈
 - mmap/munmap
 - 分配内存 buffer
 - 加载新的代码库

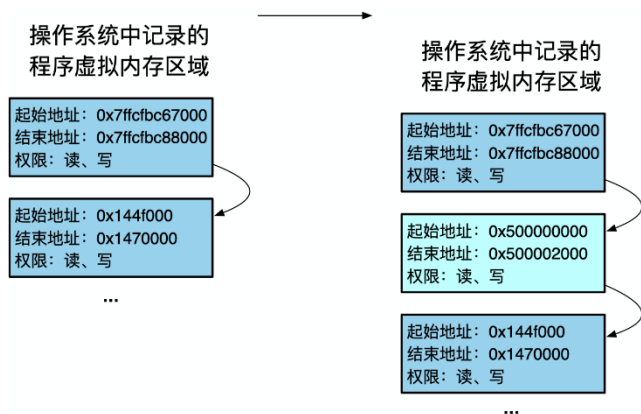
mmap: 分配一段虚拟内存区域

- 通常用于把一个文件（或部分）映射到内存 VMA 中还会包含文件映射等信息
- 也可以不映射任何文件, 仅仅新建虚拟内存区域 (匿名映射)
- 缺页异常 (page fault)
 - AARCH64: 触发 (通用的) 同步异常 (8)

- 根据 ESR 信息判断是否为缺页异常
- 访问的虚拟地址存放在 FAR_EL1

• 操作系统的缺页处理函数

- FAR_EL1 中的值不落在 VMA 区域内，则为非法
- 反之，则分配物理页，并在页表中添加映射



- 优势：节约内存资源
- 劣势：缺页异常导致访问延迟增加
- 如何取得平衡？
 - 应用程序访存具有时空局部性 (Locality)
 - 在缺页异常处理函数中采用预先映射的策略（预测相邻的虚拟页也会被访问，提前映射）
 - 在节约内存和减少缺页异常次数之间取得平衡

• madvise

- `int madvise(void *addr, size_t length, int advice)`
- 将用户态的一些语义信息发给内核以便于优化
 - 例如：将 `madvise` 和 `mmap` 搭配，在使用数据前告诉内核这一段数据需要使用，**建议** OS 提前分配物理页，减少缺页异常开销

• mprotect

- `int mprotect(void *addr, size_t len, int prot);`
- 改变一段内存的权限
 - 例如：JIT 动态生成的二进制代码，需将内存由“可写”改“为可执行”

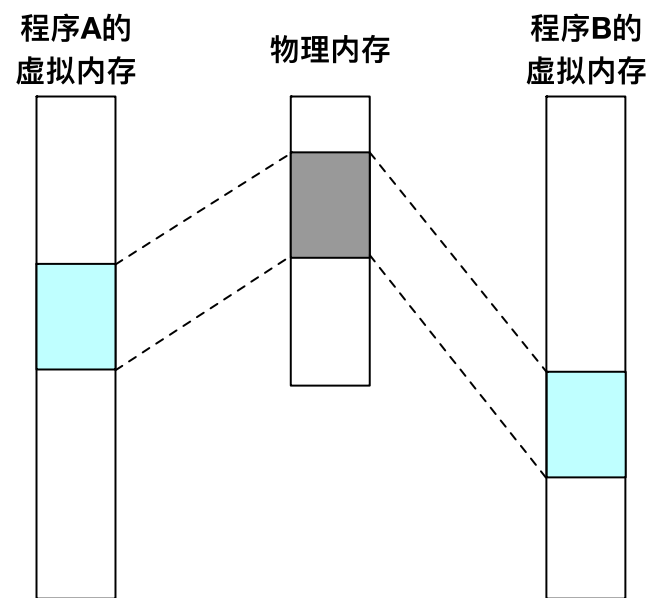
共享内存

写时拷贝 (copy-on-write)

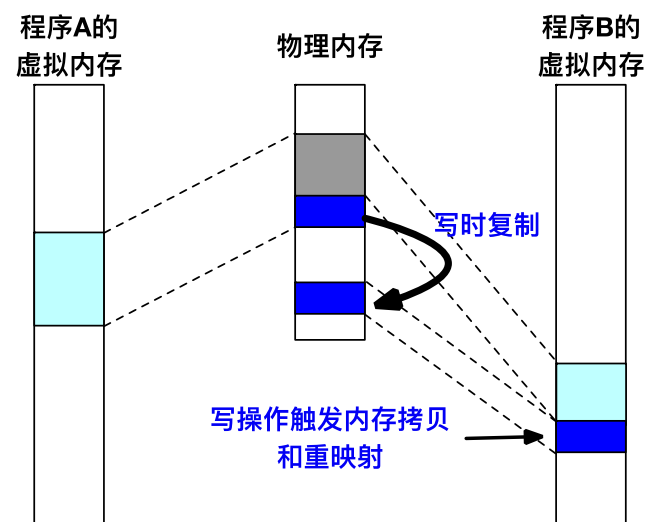
• 实现

- 修改页表项权限
- 在缺页时拷贝、恢复

• 典型场景 fork



程序A和B有一块共享内存



以写时拷贝的方式共享内存

- 节约物理内存
- 性能加速

内存去重

- memory deduplication
 - 基于写时拷贝机制
 - 在内存中扫描发现具有相同内容的物理页面
 - 执行去重
 - 操作系统发起，对用户态透明
- 典型案例：Linux KSM
 - kernel same-page merging

内存压缩

• Windows 10

- 压缩后的数据仍然存放在内存中
- 当访问被压缩的数据时，操作系统将其解压即可
- 思考：对比交换内存页到磁盘，压缩的

优点和缺点有哪些？

- Linux

- zswap: 换页过程中磁盘的缓存
- 将准备换出的数据压缩并先写入 zswap 区域 (内存)
- 好处: 减少甚至避免磁盘 I/O; 增加设备寿命

大页的利弊

- 好处

- 减少 TLB 缓存项的使用, 提高 TLB 命中率
- 减少页表的级数, 提升遍历页表的效率

- 案例

- 提供 API 允许应用程序进行显示的大页分配
- 透明大页 (Transparent Huge Pages) 机制

- 弊端

- 未使用整个大页而造成物理内存资源浪费
- 增加管理内存的复杂度

AARCH64 支持多种最小页面大小

- TCR_EL1 可以选择不同的最小页面大小
 - 3 种配置: 4K、16K、64K
 - 4K + 大页: 2M/1G
 - 16K + 大页: 32M
 - 只有 L2 页表项支持大页
 - 64K + 大页: 512M
 - 只有 L2 页表项支持大页 (ARMv8.2 之前)

物理内存

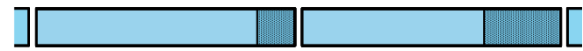
- 引入虚拟内存后, **物理内存分配**主要在以下四个场景出现:
 1. 用户态应用程序触发 on-demand paging (延迟映射) 时
 - 此时内核需要分配物理内存页, 映射到对应的虚拟页
 2. 内核自己申请内存并使用时
 - 如用于内核自身的数据结构, 通常通过 kmalloc() 完成
 3. 发生换页 (swapping) 时
 - 通过磁盘来扩展物理内存的容量
 4. 内核申请用于设备的 DMA 内存时
 - DMA 内存通常需要连续的物理页、
- 应用调用 malloc 后, 返回的虚拟地址属于某个 VMA
- 但虚拟地址对应的页表项的 valid bit 可

能为 0

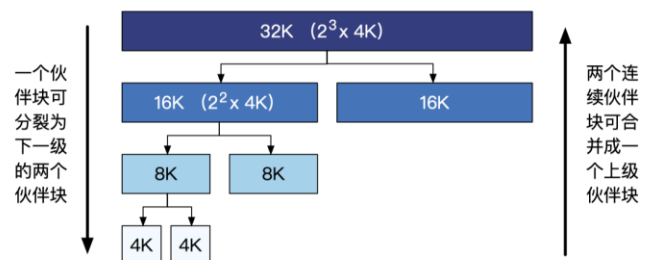
- 当第一次访问新分配的虚拟地址时, CPU 会触发 page fault



外部碎片: 单个空白部分都小于分配请求的内存大小, 但加起来足够
注: 蓝色部分表示已分配内存, 空白部分为未分配内存



内部碎片: 蓝色阴影部分是分配内存大于实际使用内存而导致的内部碎片
注: 黑色粗线框表示已分配内存, 蓝色部分表示实际使用内存, 蓝色阴影表示已分配但未使用部分



- 分配物理页/连续物理页 (2^n)

- 直接映射:

- OS 一次性将所有物理内存映射到一段高虚拟地址 (OS 使用的虚拟地址)
- 任一物理地址和 OS 使用的虚拟地址仅相差一个偏移量
- OS 可迅速根据物理地址或虚拟地址互相计算
 - 用途举例 1: OS 为应用分配物理页, 在页表中添加映射需要物理地址; OS 对物理页清零需要使用虚拟地址
 - 用途举例 2: OS 分配页表页 (即分配单个物理页), 页表项内容需要设置物理地址, 而填写页表项需要使用虚拟地址

SLAB

- 只分配固定大小块
- 对于每个固定块大小, SLUB 分配器都会使用独立的内存资源池进行分配
 - 采用 best fit 定位资源池
- Current 指向一个 slab, 并从其中分配;
- 当 Current slab 全满, 则从 Partial 链表中取出一个放入 Current
- 释放后, 若某个 slab 不再全满, 则加入 partial
- 释放后, 若某个 slab 全空则可还给伙伴系统

缺页异常

- 导致缺页异常的三种可能

- 访问非法虚拟地址
- 按需分配 (尚未分配真正的物理页)
- 内存页数据被换出到磁盘上

- 应用程序访存具有**时空局部性** (Locality)
 - 在缺页异常处理函数中采用预先映射的策略（预测相邻的虚拟页也会被访问，提前映射）
 - 在节约内存和减少缺页异常次数之间取得平衡
- 预取机制 (Prefetching)
 - 预测接卸来进程要使用的页，提前换入
 - 在缺页异常处理函数中，根据应用程序访存具有的空间本地性进行预取

理想的换页策略 (OPT 策略)

假设物理内存中可以存放三个物理页，初始为空，某应用程序一共需要访问物理页面 1 ~ 5

OPT: 优先换出未来最长时间不会再访问的页面

FIFO 策略

操作系统维护一个队列用于记录换入内存的物理页号，每换入一个物理页就把其页号加到队尾，因此最先换进的物理页号总是处于队头位置

Belady's Anomaly 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Second Chance 策略

FIFO 策略的一种改进版本：为每一个物理页号维护一个访问标志位。如果访问的页面号已经处在队列中，则置上其访问标志位。

换页时查看队头：1) 无标志则换出；2) 有标志则去除并放入队尾，继续寻找

LRU 策略

OS 维护一个链表，在每次内存访问后，OS 把刚刚访问的内存页调整到链表尾端；每次都选择换出位于链表头部的页面

缺点-1: 对于特定的序列，效果可能非常差，如循环访问内存

缺点-2: 需要排序的内存页可能非常多，导致很高的额外负载

时钟算法策略

- **精准的 LRU 策略难以实现**
- **物理页环形排列 (类似时钟)**
 - 为每个物理页维护一个访问位
 - 当物理页被访问时，把访问位设成 T
 - OS 依次 (如顺时针) 查看每个页的“访问位”
 - 如果是 T，则置成 F
 - 如果是 F，则驱逐该页
- 时钟算法 (Clock Algorithm) 将换进物理内存的页号排成一个时钟的形状。该时钟有一个针臂，指向新换进内存的页号的后一个。同时，也为每一个页号维护一个访问标志位。每次需要选择换出页号时，该算法从针臂所指的页号开始检查。如果当前页面的访问标志位没有设

置，即从上次检查到这次，该页面没有被访问过，将该页面替换；如果当前页面的访问标志位已被置上 (即被访问过)，那就将其访问标志位清空，并且针臂顺时针移动指针到下一个页面。如此重复，直到找到一个访问标志位未被置上的页面。

物理页面访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
	3*	3*	3*	3*	4*	4*	4*	4*	2*	2*	2*	2*
		2*	2*	2*	2	3*	3*	3*	3	3*	3	3*
				1*	1	1	5*	5*	5	5	4*	4*
缺页异常	是	是	否	是	是	是	是	否	是	否	是	否

- **每个物理页需要有一个“访问位”**
 - MMU 在页表项里面为**虚拟页**打上“访问位”
 - 回顾：页表项中的 Access Flag
- **如何实现**
 - OS 在描述物理页的结构体里面记录页表项位置
 - 当物理页被填写到某张页表中时，把页表项的位置记录在元数据中 (在 Linux 中称为“反向映射”：**reverse mapping**)
 - 根据物理页对应的页表项中的“访问位”判断是否驱逐
 - 驱逐某页时应该清空其对应的所有页表项 (例如共享内存)

Thrashing Problem

- **直接原因**
 - 过于频繁的缺页异常 (物理内存总需求过大)
- 大部分 CPU 时间都被用来处理缺页异常
 - 等待缓慢的磁盘 I/O 操作
 - 仅剩小部分的时间用于执行真正有意义的工作
- 调度器造成问题加剧
 - 等待磁盘 I/O 导致 CPU 利用率下降
 - 调度器载入更多的进程以期提高 CPU 利用率
 - 触发更多的缺页异常、进一步降低 CPU 利用率、导致连锁反应

工作集模型 (有效避免 Thrashing)

- 一个进程在时间 t 的工作集 W(t, x):
 - 其在时间段 (t - x, t) 内使用的内存页集合
 - 也被视为其在未来 (下一个 x 时间内) 会访问的页集合
 - 如果希望进程能够顺利进展，则需要将该集合保持在内存中
- 工作集模型: All-or-nothing
 - 进程工作集要么都在内存中，要么全都

换出

- 避免 thrashing, 提高系统整体性能表现

五、通信与调度

进程切换

- 操作系统为每个进程维护处理器上下文
 - 包含恢复进程执行所需要的状态
 - 思考: 进程 A 执行到 main 函数任意一条指令, 切换到进程 B 执行, 一段时间后, 再切回到进程 A 执行
 - 为完成此过程, 有哪些状态需要保存?
 - 具体包括:
 - **PC 寄存器值, 栈寄存器值, 通用寄存器值, 状态寄存器值**

- **进程 ID**
 - 每个进程都有唯一的正数 PID
- **getpid()**
 - 返回调用进程的 PID
- **getppid()**
 - 返回调用进程父进程的 PID
 - 父进程: 创建该进程的进程

getpid 和 getppid 函数返回 pid_t 类型的 int 值
exit 函数终止进程并带上一个 status 状态

Fork 函数

- 调用一次
 - 在父进程中
- 返回两次
 - 在父进程中, 返回子进程的 PID
 - 在子进程中, 返回 0
- 返回值提供了**唯一**明确地区分父进程和子进程执行的方法

Linux 下的僵尸进程

- 进程终止后, 内核不会立刻销毁该进程
 - 不再运行, 但仍然占用内存资源
- 进程以终止态存在, 等待父进程回收
- 当父进程回收终止的子进程
 - 内核把子进程的 exit 状态传递给父进程
 - 内核移除子进程, 此时子进程才被真正回收
- 终止状态下还未被回收的进程就是僵尸进程
- 如果父进程
 - 在自己终止前没有回收僵尸子进程
 - 内核会安排 init 进程回收这些子进程
- **init 进程**
 - PID 为 1
 - 在系统初始化时由内核创建

waitpid 函数

- pid>0: 等待集合中只有 pid 子进程

- pid=-1: 等待集合包括所有子进程
- 如果没有子进程: 返回-1, errno = ECHILD
- 如果等待被中断: 返回-1, errno = EINTR

进程

- 进程是计算机程序运行时的抽象
 - 静态部分: 程序运行需要的代码和数据
 - 动态部分: 程序运行期间的状态 (程序计数器、堆、栈……)
- **进程具有独立的虚拟地址空间**
 - 每个进程都具有“独占全部内存”的假象
 - 内核中同样包含内核栈和内核代码、数据

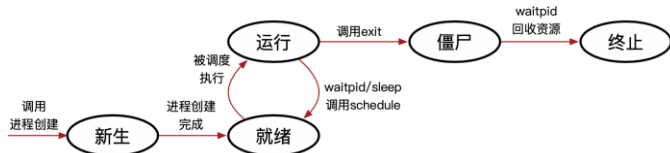
每个进程都对应一个元数据, 称为“进程控制块”PCB

- **调度队列**
 - 队列中每个元素是进程 PCB
- **内核调度器从调度队列中取出一个进程 PCB**
 - schedule(): 进程切换
 - 恢复其上下文状态
 - 通过 eret 指令恢复其执行

PCB 创建

- **PCB 及其包含的内容都需要创建及初始化**
 - 分配 PCB 本身的数据结构
 - 初始化 PCB: 虚拟内存
 - 创建及初始化 *vmSPACE* 数据结构
 - 分配一个物理页, 作为顶级页表
 - 内核栈: 分配物理页, 作为进程内核栈
- **可执行文件通常有固定的存储格式**
- **以 ELF (Executable and Linkable Format) 为例**
 - 从程序头部表可以获取需要的段所在位置
 - 通常只有代码段和数据段需要被加载 (loadable)
 - 加载即从 ELF 文件中映射到虚拟地址空间的过程
- **在返回用户态运行前, 还需为进程准备运行所需的环境**
 - 分配用户栈 (分配物理内存并映射到虚拟地址空间)
 - 准备程序运行时的环境
 - 回顾 main 函数: int main(int argc, char* argv[], char *envp[])
- **为什么直到最后才初始化处理器上下文?**
 - 其包含的内容直到前序操作完成才确定
 - SP: 用户栈分配后才确定地址
 - PC (保存在 ELR_EL1): 加载 ELF 后才知道入口所在地址
 - 通用寄存器初始值可直接赋为 0

- PSTATE, 会有一个新进程执行前的初始处理器状态, 但不能直接赋值给 PSTATE, 需要赋值给 SPSR_EL1



```

1 enum exec_status {NEW, READY, RUNNING,
2                  ZOMBIE, TERMINATED};
3
4 struct process_v5 {
5     // 处理器上下文
6     struct context *ctx;
7     // 虚拟地址空间 (包含页表基地址)
8     struct vmSPACE *vmSPACE;
9     // 内核栈
10    void *stack;
11    // 进程标识符
12    int pid;
13    // 退出状态
14    int exit_status;
15    // 子进程列表
16    pcb_list *children;
17    // 执行状态
18    enum exec_status exec_status;
19 };

```

Fork()

- 语义: 为调用进程创建一个一模一样的新进程
 - 调用进程为父进程, 新进程为子进程
 - 接口简单, 无需任何参数
- fork 后的两个进程均为独立进程
 - 拥有不同的进程 id
 - 可以并行执行, 互不干扰
 - 父进程和子进程会共享部分数据结构 (例如文件描述符)

Fork 后父子进程顺序不确定, 视调度策略而定

- 为了“一模一样”, 将父进程的 PCB 拷贝一份包括 PCB、内核栈、处理器上下文
- fork 的优点
 - 接口非常简洁, (过去) 实现简单
 - 将进程“创建”和“执行” (exec) 解耦, 提高了灵活性
- fork 的缺点
 - 创建拷贝的复杂度与 PCB 复杂度相关 (如今越来越复杂)
 - 完全拷贝过于粗暴 (不如 clone)
 - 性能差、可扩展性差 (不如 vfork 和 spawn)
 - 不可组合性 (例如: fork() + pthread())
- vfork: 类似于 fork, 但让父子进程共享同一地址空间
 - 优点: 连映射都不需要拷贝, 性能更好
 - 缺点:
 - 通常只能用在“fork + exec”的场景中 (vfork 调用会暂停父进程

执行)

- 共享地址空间存在安全问题
- posix_spawn: 相当于 fork + exec
 - 优点: 可扩展性、性能较好
 - 缺点: 不如 fork 灵活
- clone: fork 的“进阶版”, 可以选择性地不拷贝内存
 - 优点: 高度可控, 可依照需求调整
 - 缺点: 接口比 fork 复杂, 选择性拷贝容易出错

进程切换

- 需要进出内核, 共包含五个步骤
 1. 进程 1 进入内核态
 2. 进程 1 处理器上下文保存
 3. 进程上下文切换
 4. 进程 2 处理器上下文恢复
 5. 进程 2 返回用户态
- 处理器上下文: 用于保存切换时的寄存器状态 (硬件)
 1. 在每个 PCB 中均有保存
- 进程上下文: 表示目前操作系统正以哪个进程的身份运行 (软件)
 1. 通常使用一个指向 PCB 的全局指针表示 (代码中的 curr_proc)

所有调用 schedule() 的地方都涉及进程切换

(1): p0 进入内核态

- 由硬件完成部分寄存器保存
 - PC 和 PSTATE 分别自动保存到 ELR_EL1 和 SPSR_EL1

(2): p0 处理器上下文保存

- 将处理器中的寄存器值保存到处理器上下文对应的位置

(3.1): 虚拟地址空间切换

- 设置页表相关寄存器 (TTBR0_EL1)
 - 使用 PCB 中保存的页表基地址赋值给 TTBR0_EL1

(3.2): 内核栈切换

- 设置内核中的栈寄存器 SP_EL1
 - 使用 PCB 中保存的内核栈顶地址赋值给 SP_EL1

(3.3): 进程上下文切换

- 设置 cur_proc 为之后要执行的进程 (p1)
 - 表明之后操作系统将以 p1 的身份运行

(4): p1 处理器上下文恢复

- 从处理器上下文中加载各寄存器的值, 放入对应寄存器中

(5): p1 处理器上下文恢复

- 由硬件自动恢复部分寄存器
 - 将 ELR_EL1 和 SPSR_EL1 中的值自动保

存到 PC 和 PSTATE 中

线程

Fork 方法存在的局限

- 进程间隔离过强，共享数据困难
 - 各进程拥有独立的地址空间，共享需以页为粒度
 - 协调困难，需要复杂的通信机制
- 进程管理开销较大
 - 创建：地址空间复制
 - 切换：页表切换
- 线程只包含运行时的状态
 - 代码和数据等由进程提供
 - 包括了执行所需的最小状态（主要是寄存器和栈）
- 一个进程可以包含多个线程
 - 每个线程共享同一地址空间（方便数据共享和交互）
 - 允许进程内并行

一个线程执行系统调用，可能影响该进程的所有线程

pthread 使用

- 常用接口：创建、合并、分离、退出
- 线程资源默认需要显示回收
 - 主线程可使用合并（pthread_join）回收其他线程
 - 线程也可主动调用分离函数（pthread_detach）以自动回收
- 主线程退出默认会终结所有线程
 - 可改为调用退出（pthread_exit），只退出主线程

多线程进程的地址空间

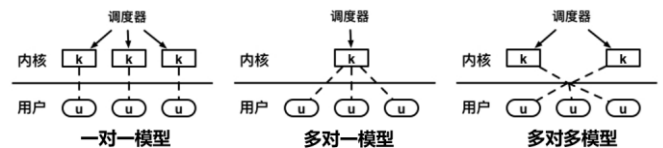
- 每个线程拥有自己的栈
- 内核中也有为线程准备的内核栈
- 其他区域共享
 - 数据、代码、堆……

用户态线程与内核态线程

- 根据线程是否受内核管理，可以将线程分为两类
 - 内核态线程：内核可见，受内核管理
 - 用户态线程：内核不可见，不受内核直接管理
- 内核态线程
 - 由内核创建，线程相关信息存放在内核中
- 用户态线程
 - 在应用态创建，线程相关信息主要存放在应用数据中
- 线程模型表示了用户态线程与内核态线程之间的联系
 - 多对一模型：多个用户态线程对应一个

内核态线程

- 一对一模型：一个用户态线程对应一个内核态线程
- 多对多模型：多个用户态线程对应多个内核态线程



多对一模型

- 将多个用户态线程映射给单一的内核线程
 - 优点：内核管理简单
 - 缺点：可扩展性差，无法适应多核机器的发展
 - 在主流操作系统中被弃用
 - 用于各种用户态线程库中（协程）

一对一模型

- 每个用户线程映射单独的内核线程
 - 优点：解决了多对一模型中的可扩展性问题
 - 缺点：线程切换需要经过内核
- 主流操作系统都采用一对一模型
 - Windows、Linux、OS X……

多对多模型

- N 个用户态线程映射到 M 个内核态线程 (N > M)
 - 优点：解决可扩展性问题（多对一）和线程切换时延问题（一对一）
 - 缺点：管理更为复杂
 - 在虚拟化中得到了广泛应用

进程 vs. 线程

- 线程和进程的相似之处：
 - 都可以与其他进程/线程并发执行（可能在不同核心上）
 - 都可以进行切换
 - 引入线程后，调度管理单位由进程变为线程
- 线程和进程的不同之处：
 - 同一进程的不同线程共享代码和部分数据
 - 不同进程不共享虚拟地址空间
 - 线程与进程相比开销较低
 - 进程控制（创建和回收）通常比线程更耗时
 - Linux 的数据：
 - 创建和回收进程：~ 20K cycle
 - 创建和回收线程：~ 10K cycle（或更少）

TCB (线程控制块)

- 将 PCB 中部分内容移入 TCB 中
 - 每个线程 TCB 保存自己的处理器上下文、内核栈、退出/执行状态
 - 进程 PCB 仍维护共享的地址空间
 - PCB/TCB 间相互引用, 便于管理
 - 与进程创建相比步骤更少 (如不需要加载可执行文件)
 - 在 Linux 中, 线程创建由 clone (本用于创建进程) 实现

Sidebar: 多线程 fork

- 一个多线程的程序调用 fork, 会发生什么?
- 实现方式 1: 拷贝父进程中所有的线程
- 实现方式 2: 只拷贝父进程中调用 fork 的线程
- POSIX 建议: 尽量避免用 fork 拷贝多线程程序
 - 若希望使用多进程: 使用 fork
 - 若希望使用多线程: 直接使用 pthread_create

调度

- **降低周转时间:** 任务第一次进入系统到执行结束的时间
- **降低响应时间:** 任务第一次进入系统到第一次给用户输出的时间
- **实时性:** 在任务的截止时间内完成任务
- **公平性:** 每个任务都应该有机会执行, 不能饿死
- **开销低:** 调度器是为了优化系统, 而非制造性能 BUG
- **可扩展:** 随着任务数量增加, 仍能正常工作

First Come First Served

先到先得: 简单、直观

问题: 平均周转、响应时间过长

Shortest Job First

短任务优先: 平均周转时间短

问题: 1) 不公平, 任务饿死

2) 平均响应时间过长

STCF (Shortest Time-to-Completion First) 最短完成时间优先

Round Robin (时间片轮转)

固定时间顺序切换

轮询: 公平、平均响应时间短

问题: 牺牲周转时间

优先级调度

Multi-Level Queue (MLQ)

- 1) 维护多个队列, 每个队列设置不同优先级
- 2) 高优先级队列中的任务优先执行
- 3) 同优先级内使用 Round Robin 调度 (也可使用其他调度策略)

$$\text{优先级} = \frac{T_{\text{Waiting}}}{T_{\text{Run}}}$$

- 如果两个任务等待时间 (T_{Waiting}) 相同, 则运行时间越短越优先
- 如果两个任务运行时间相同, 则等待时间越长, 越优先
- 结合 FCFS 策略和 SJF 策略, 避免了 SJF 策略在公平性方面的问题

MLFQ: 多级反馈队列

- 规则 1:
 - 优先级高的任务会抢占优先级低的任务
- 规则 2:
 - 每个任务会被分配时间片, 优先级相同的两个任务使用时间片轮转
- 规则 3:
 - 任务被创建时, 假设该任务是短任务, 为它分配最高优先级
- 规则 4a:
 - 一个任务时间片耗尽后, 它的优先级会被降低一级
- 规则 4b:
 - 如果一个任务在时间片耗尽让出 CPU, 那么它的优先级不变
 - (通常, 交互式任务/IO 密集型任务符合该特点)
 - 任务重新执行时, 会被分配新的时间片

解决饿死

- 规则 5:
 - 在某个时间段 S 后, 将系统中所有任务优先级升为最高
- 效果 1: 避免长任务饿死
 - 所有任务的优先级会定时地提升最高
 - 最高级队列采用 RR, 长任务一定会被调度到
- 效果 2: 针对任务特征动态变化的场景
 - MLFQ 会定时地重新审视每个任务

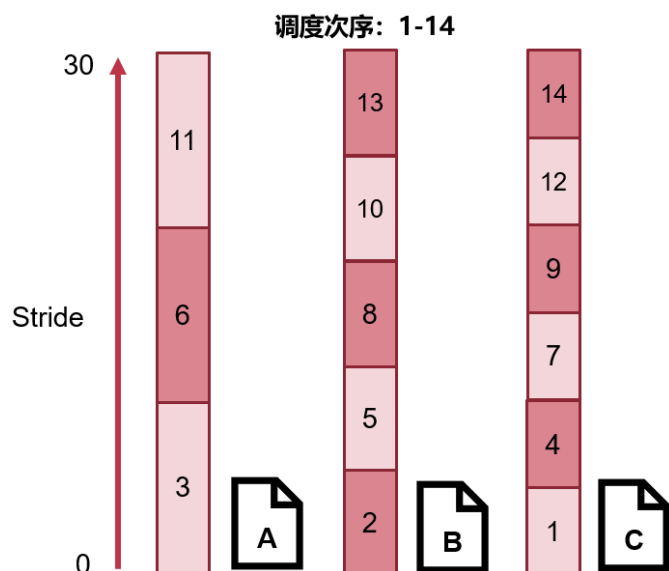
解决攻击

- 规则 4:
 - 一个任务时间片耗尽后 (无论它期间放弃了多次 CPU, 它的时间片不会被重置), 它的优先级会被降低一级
- 更新策略
 - 记录每个任务在当前优先级使用的时间片
 - 当累计一个完整时间片被用完后, 降低其优先级

为不同队列选择不同的时间片

- 高优先级队列时间片较短, 针对短任务
 - 提升响应时间

- 低优先级队列时间片较长，针对长任务
 - 降低调度开销



方法：使用“ticket”表示任务的份额

ticket：每个问题对应的份额

T ：ticket 的总量

问题 A1 可占用学霸时间的比例

$$\frac{\text{ticket}_{A1}}{T} = \frac{20}{100} = \frac{1}{5}$$

同学 A 可占用学霸时间的比例

$$\frac{\text{ticket}_A}{T} = \frac{\text{ticket}_{A1} + \text{ticket}_{A2}}{T} = \frac{50}{100} = \frac{1}{2}$$

一种公平共享的实现：彩票调度 (Lottery Scheduling)

- 每次调度时，生成随机数 $R \in [0, T)$
- 根据 R ，找到对应的任务
 - $R=51 \rightarrow$ 调度 C

步幅调度 (Stride Scheduling)

- 确定性版本的 Lottery Scheduling
 - 可以沿用 tickets 的概念
- Stride——步幅，任务一次执行增加的虚拟时间

$$\text{stride} = \frac{\text{MaxStride}}{\text{ticket}}$$

- MaxStride 是一个足够大的整数
- 本例中设为所有 tickets 的最小公倍数

- Pass——累计执行的虚拟时间

负载均衡 (Load Balance)

- 需要追踪 CPU 的负载情况
- 将任务从负载高的 CPU 迁移到负载低的 CPU

进程间通信

管道 pipe

- 管道是 Linux 等系统中常见的进程间通信机制
 - 两个进程间的一根通信通道（无格式的数据）

- 一端向里投递，另一端接收
- 命令行常用：常见的命令 `ls | grep`
- Linux 系统调用
 - `pipe()` 用于创建一个匿名管道
 - `mkfifo()` 用于创建一个命名管道

消息队列：带类型的消息传递

- 消息队列：以链表的方式组织消息
 - 信箱为内核中维护的消息队列结构体
 - 任何有权限的进程都可以访问队列，写入或者读取
 - 支持异步通信（非阻塞）
- 消息的格式：类型 + 数据
 - 类型：由一个整型表示，具体的意义由应用程序决定
- 消息队列的组织
 - 基本遵循 FIFO (First-In-First-Out) 先进先出原则
 - 消息队列的写入：增加在队列尾部
 - 消息队列的读取：默认从队首获取消息
- 允许按照类型查询：`Recv(A, type, message)`
 - 类型为 0 时返回第一个消息 (FIFO)
 - 类型有值时按照类型查询消息
 - 如 `type` 为正数，则返回第一个类型为 `type` 的消息

消息队列 VS. 管道

- 消息格式：
 - 消息队列：带类型的数据
 - 管道：数据（字节流）
- 通信进程数量：
 - 消息队列：可以有多个发送者和接收者
 - 管道：两个端口，最多对应两个进程
- 消息的管理：
 - 消息队列：FIFO + 基于类型的查询
 - 管道：FIFO
- 缓存区设计：
 - 消息队列：链表的组织方式，动态分配资源，可以设置很大的上限
 - 管道：固定的缓冲区间，分配过大资源容易造成浪费

共享内存通信

- 缺少通知机制
 - 若轮询检查，则导致 CPU 资源浪费
 - 若周期性检查，则可能导致较长的等待时延
 - 根本原因：共享内存的抽象过于底层；缺少 OS 更多支持
- TOCTTOU (Time-of-check to Time-of-use) 问题
 - 当接收者直接用共享内存上的数据时，

可能存在被发送者恶意篡改的情况（发生在接收者检查完数据之后，使用数据之前）

- 这可能导致 buffer overflow 等问题

IPC 的基本功能目标

- 数据传递
 - 发送消息：Send
 - 接收消息：Recv
 - 例如：pipe, msgqueue, 网络 socket, 消息订阅发布 publish/subscribe
- 过程调用
 - 包括数据传递
 - 远程方法调用：RPC
 - 调用结果返回：Reply
 - 例如：进程 1 调用进程 2 的方法（与远程过程调用 RPC 类似）
- 阶段-1：准备阶段
 - 建立通信连接，即进程间的信道
 - 假设内核已经为两个进程映射了一段共享内存
- 阶段-2：通信阶段
 - 数据传递
 - “消息”抽象：通常包含头部（元数据）和数据内容（例如 500 字节）
 - 通信机制
 - 两个消息保存在共享内存中：发送者消息、接收者消息
 - 发送者和接收者通过轮询消息的状态作为通知机制
- 方法-1：通过共享内存的数据传递
 - 操作系统在通信过程中不干预数据传输
 - 操作系统仅负责准备阶段的映射
- 方法-2：通过操作系统内核的数据传递
 - 操作系统提供接口（系统调用）
 - 通过内核态内存来传递数据，无需在用户态建立共享内存
- 基于共享内存的优势
 - 完全由用户态控制，定制能力更强
 - 无需内核进行额外的内存拷贝
- 基于系统调用的优势
 - 抽象更简单，用户态直接调用接口，使用更方便
 - 安全性保证更强，发送者在消息被接收时通常无法修改消息

简单 IPC 的通知机制

- 方法-1：基于轮询（消息头部的状态信息）
 - 缺点：大量 CPU 计算资源的浪费
- 方法-2：基于控制流转移

- 由内核控制进程的运行状态
- 优点：进程只有在条件满足的情况下才运行，避免 CPU 浪费

同步 IPC

- IPC 操作会阻塞进程直到操作完成
- 线性的控制流
- 调用者继续运行时，返回结果已经就绪

异步 IPC

- 进程发起 IPC 操作后即可返回而不需要等待其完成
- 通过轮询或回调函数（需内核支持）来获取返回结果

IPC 的两种通信连接

方法-1：直接通信

- 通信的一方需要显示地标识另一方，每一方都拥有唯一标识
- 如：Send(P, message), Recv(Q, message)
- 连接的建立是自动完成的（由内核完成）

方法-2：间接通信

- 通信双方通过“信箱”的抽象来完成通信
- 每个信箱有自己唯一的标识符
- 通信双方并不直接知道在与谁通信
- 进程间连接的建立发生在共享一个信箱时

	IPC机制	数据抽象	参与者	方向	内核实现
	管道	字节流	两个进程	单向	通常以FIFO的缓冲区来管理数据。有匿名管道和命名管道两类主要实现。
	消息队列	消息	多进程	单向 双向	队列的组织方式。通过文件的权限来管理对队列的访问。
	信号量	计数器	多进程	单向 双向	内核维护共享计数器。通过文件的权限来管理对计数器的访问。
	共享内存	内存区间	多进程	单向 双向	内核维护共享的内存区间。通过文件的权限来管理对共享内存的访问。
	信号	事件编号	多进程	单向	为线程/进程维护信号等待队列。通过用户/组等权限来管理信号的操作。
	套接字	数据报文	两个进程	单向 双向	有基于IP/端口和基于文件路径的寻址方式。利用网络栈来管理通信。

六、同步与文件

同步原语（Synchronization Primitives）是一个平台

(如操作系统) 提供的用于帮助开发者实现线程之间同步的软件工具

编程抽象：临界区 (Critical Section)

任意时刻，有且只有一个线程可以进入临界区执行

互斥锁的接口：拿锁和放锁

- 互斥锁 (Mutual Exclusive Lock) 接口
 - Lock(lock): 尝试拿到锁“lock”
 - 若当前没有其他线程拿着 lock，则拿到 lock，并继续往下执行
 - 若 lock 被其他线程拿着，则不断循环等待放锁 (busy loop)
 - Unlock(lock)
 - 释放锁
- 保证同时只有一个线程能够拿到锁

条件变量

条件变量：利用睡眠/唤醒机制，避免无意义的等待等待的接口：

void cond_wait(struct cond *cond, struct lock *mutex);

- 放入条件变量的等待队列
- 阻塞自己同时释放锁：调度器可以调度到其他线程
- 被唤醒后重新获取锁

唤醒的接口：

void cond_signal(struct cond *cond);

- 检查等待队列
- 如果有等待者则移出等待队列并唤醒

等待空位代码

```
1. ...
2. /* Wait empty slot */
3. lock(empty_cnt_lock);
4. while (empty_slot == 0)
5.     cond_wait(empty_cond,
6.               empty_cnt_lock);
7. empty_slot--;
8. unlock(empty_cnt_lock);
9. ...

1. ...
2. /* Wait empty slot */
3. lock(empty_cnt_lock);
4. while (empty_slot == 0)
5.     unlock(empty_cnt_lock);
6.     cond_wait_nolock(empty_cond)
7. empty_slot--;
8. unlock(empty_cnt_lock);
9. ...
```

若5-6行之间收到信号通知，
则存在信号丢失问题！

线程 1

线程 2

```
lock(empty_cnt_lock);
if (empty_slot == 0)
    时刻1 cond_wait(empty_cond,
                    empty_cnt_lock);

    时刻4 错误唤醒！
empty_slot--;
unlock(empty_cnt_lock);
empty_slot = -1
```

时刻2

有新的空位，唤醒
empty_slot = 1

```
lock(empty_cnt_lock);
empty_slot--;
unlock(empty_cnt_lock);...
empty_slot = 0
    时刻3 重新拿到锁
```

信号量 (Semaphore) (PV 原语)

信号量：协调 (阻塞/放行)

多个线程共享有限数量的资源

语义上：信号量的值 cnt 记录了当前可用资源的数量

提供了两个原语 P 和 V 用于等待/消耗资源

P 操作：消耗资源

```
void sem_wait(sem_t *sem) {
    while(sem->cnt <= 0)
        /* Waiting */;
    sem->cnt--;
```

}

V 操作：增加资源

```
void sem_signal(sem_t *sem) {
    sem->cnt++;
}
```

二元信号量与计数信号量

```
void producer(void) {
    new_msg = produce_new();
    sem_wait(&empty_slot_sem);      消耗empty_slot
    buffer_add(new_msg);
    sem_signal(&filled_slot_sem);    增加filled_slot
}

void consumer(void) {
    sem_wait(&filled_slot_sem);      消耗filled_slot
    cur_msg = buffer_remove();
    sem_signal(&empty_slot_sem);      增加empty_slot
    handle_msg(cur_msg);
}
```

当初始化的资源数量为 1 时，为二元信号量

其计数器 (counter) 只有可能为 0、1 两个值，故被称为二元信号量

同一时刻只有一个线程能够拿到资源

当初始化的资源数量大于 1 时，为计数信号量

同一时刻可能有多个线程能够拿到资源

读写锁

互斥锁：所有的线程均互斥，同一时刻只能有一个线程进入临界区

对于部分只读取共享数据的线程过于严厉

读写锁：区分读者与写者，允许读者之间并行，读者与写者之间互斥

同步原语对比：互斥锁/条件变量/信号量

- 互斥锁与二元信号量功能类似，但抽象不同：
 - 互斥锁有拥有者的概念，一般同一个线程拿锁/放锁
 - 信号量为资源协调，一般一个线程 signal，另一个线程 wait
- 条件变量用于解决不同问题 (睡眠/唤醒)，需要搭配互斥锁使用

同步原语对比：互斥锁 vs 读写锁

- 接口不同：读写锁区分读者与写者
- 针对场景不同：获取更多程序语义，标明只读代码段，达到更好性能
- 读写锁在读多写少场景中可以显著提升读者并行度
 - 即允许多个读者同时执行读临界区
- 只用写者锁，则与互斥锁的语义基本相同

死锁预防：方法一

只有代理线程能够访问共享资源：避免数据竞争

死锁预防：方法二

不允许持有锁并等待拿其他锁：若拿不到某把锁，则把已获取的锁也放了

死锁预防：方法三

打破循环等待：按照特定顺序获取锁

- 对锁进行编号
- 让所有线程递增获取

任意时刻：获取最大资源号的线程可以继续执行，然后释放资源

新指令-1: Test-and-Set

```
1 int TestAndSet(int *old_ptr, int new) {
2   int old = *old_ptr; // fetch old value at old_ptr
3   *old_ptr = new; // store 'new' into old_ptr
4   return old; // return the old value
5 }
```

新指令-2: Compare-and-swap

```
1 int CompareAndSwap(int *ptr, int expected, int new) {
2   int actual = *ptr;
3   if (actual == expected)
4     *ptr = new;
5   return actual;
6 }
```

硬件方法：用原子指令

新指令-3: Load-linked & Store-conditional

```
1 int LoadLinked(int *ptr) {
2   return *ptr;
3 }
4
5 int StoreConditional(int *ptr, int value) {
6   if (no one has updated *ptr since the LoadLinked
to this address) {
7     *ptr = value;
8     return 1; // success!
9   } else {
10    return 0; // failed to update
11  }
12 }
```

用 LL/SC 来实现 Spinlock

```
1 void lock(lock_t *lock) {
2   while (1) {
3     while (LoadLinked(&lock->flag) == 1)
4       ; // spin until it's zero
5     if (StoreConditional(&lock->flag, 1) == 1)
6       return; // if set-it-to-1 was a success: all done
7       // otherwise: try it all over again
8   }
9 }
10
11 void unlock(lock_t *lock) {
12   lock->flag = 0;
13 }
```

新指令-4: Fetch-and-add

```
1 int FetchAndAdd(int *ptr) {
2   int old = *ptr;
```

```
3   *ptr = old + 1;
4   return old;
5 }
```

用 Fetch-and-add 实现 Ticket Lock

```
1 typedef struct __lock_t {
2   int ticket;
3   int turn;
4 } lock_t;
5
6 void lock_init(lock_t *lock) {
7   lock->ticket = 0;
8   lock->turn = 0;
9 }
10
11 void lock(lock_t *lock) {
12   int myturn = FetchAndAdd(&lock->ticket);
13   while (lock->turn != myturn)
14     ; // spin
15 }
16
17 void unlock(lock_t *lock) {
18   lock->turn = lock->turn + 1;
19 }
```

排号锁 (Ticket Lock)

通过遵循竞争者到达的顺序来传递锁。

读写锁的偏向性

- 考虑这种情况：
 - t0: 有读者在临界区
 - t1: 有新的写者在等待
 - t2: 另一个读者能否进入临界区？
- 不能：偏向写者的读写锁
 - 后序读者必须等待写者进入后才进入
- 能：偏向读者的读写锁
 - 后序读者可以直接进入临界区

同步原语	描述	使用场景
互斥锁	保证对共享资源的互斥访问	场景一 共享资源互斥访问
读写锁	允许读者线程并发读取共享资源	衍生场景一 读写场景并发读取
条件变量	提供线程睡眠与唤醒机制	场景二 条件等待与唤醒
信号量	协调有限数量资源的消耗与释放	场景三 多资源协调管理

死锁避免：银行家算法

银行家算法的核心：

- 所有线程获取资源需要通过管理者同意
- 管理者预演会不会造成死锁
 - 如果会造成：阻塞线程，下次再给
 - 如果不会造成：给线程该资源
- 全局可利用资源：Available[M]

- 每线程最大需求量：Max[N][M]
- 已分配资源：Allocation[N][M]
- 还需要的资源：Need[N][M]
- 全局可利用资源：该向量代表某一时刻系统中每一类元素的可用个数。这个向量初始化时设置为系统中拥有的第M\$类资源的总量。
- 每线程最大需求量：该矩阵包含所有线程N\$对第M\$类资源的最大需求量。
- 已分配资源：该矩阵包含已经分配给所有线程N\$的第M\$种资源的数量。
- 还需要的资源：该矩阵包含所有线程N\$对第M\$累资源还需要的资源数量。

对于一组线程 {P1, P2, ..., Pn}:

- 安全状态
 - 能找出至少一个执行序列，如P2->P1->P5...让所有线程需求得到满足
- 非安全状态
 - 不能找出这个序列，必定会导致死锁

安全性检查算法

银行家算法：保证系统一直处于安全状态，且按照这个序列执行

文件和文件系统

- 文件是对数据的一种抽象
 - 文件的定义：有名字且持久化的一段数据
- 文件系统
 - 提供了一组操作文件的 API

inode：记录文件多个磁盘块的位置

- 引入 inode：index node
 - 记录多个磁盘块号
 - 头部记录文件 size 信息
 - 每个文件对应一个 inode
 - 称为文件元数据（Metadata）
- 文件读写操作
 - 给定 inode 和文件内偏移（offset）
 - 根据 offset 计算出对应的磁盘块号
 - 若 offset 超出 size 则返回错误
- inode 表：记录所有 inode
 - 可以看成 inode 的大数组
 - 每个 inode 使用作为索引
 - 此时，inode 号即为文件名
- inode 分配信息（位图）
 - 记录哪些 inode 已分配，哪些空闲
- 超级块：Super Block
 - 记录磁盘块的大小、其他信息的起始磁盘块位置，等等
 - 是整个文件系统的元数据
- 加载文件系统
 - 首先读取超级块，然后找到其他信息
- 创建新文件
 - 根据 inode 分配信息找到空闲 inode，

将 inode 对应的 bit 设置为 1

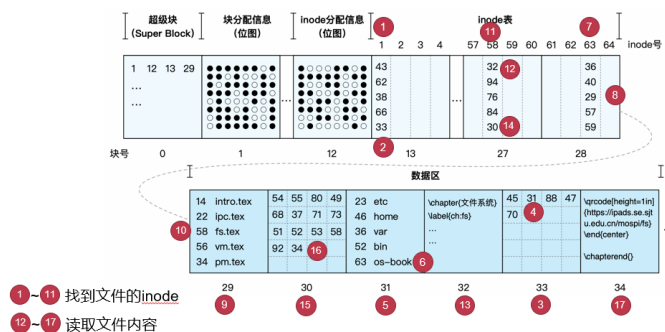
- 返回 inode 在 inode 表中的索引，作为文件名

查找文件（根据 inode 号）

- 在 inode 表中根据 inode 号定位该 inode

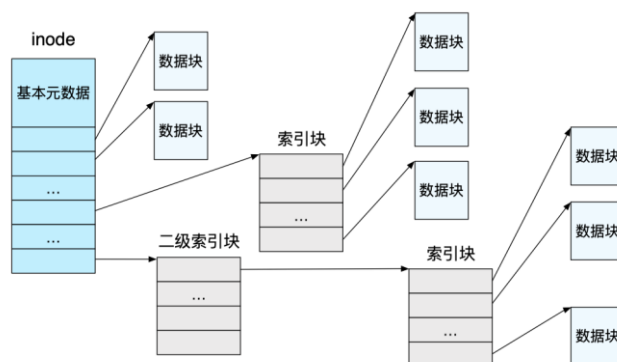
删除文件

- 在 inode 分配表中，将该 inode 对应的 bit 设置为 0



多级 inode

- 引入索引块：指向数据块；以及二级索引块：指向索引块；...
- 索引块（包括二级索引块）不在 inode 表的存储区域，而是在数据区域



目录文件与目录项

- 目录中的每条映射称为一个目录项
 - 每一条目录项记录了一个 inode 号与文件名字字符串的映射
 - 一个目录可以记录很多目录项
- 目录文件的大小（占用空间）
 - 与其记录的文件大小无关
 - 思考：与什么因素有关？
- 目录支持查找操作
 - 给定一个目录文件和字符串

- 在目录文件中查找字符串，并返回对应的 inode

• 目录中可以记录子目录

- 因为目录本身也是一个文件
- 通过“/”来分割父目录和子目录

• 最顶端的目录没有目录名（文件名）

- 被称为“根目录”（root）
- 根目录没有文件名，在“/”的前面什么都没有
- 通常固定为 1 号 inode

• 绝对路径和相对路径

- 绝对路径：如“/home/OS/test.md”
- 相对路径：如“./test.md”或“OS/test.md”

（硬）链接：Link

• LINK

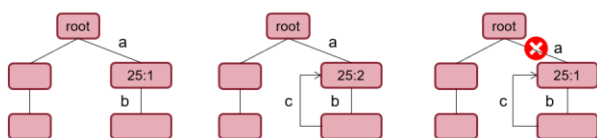
- LINK("Mail/inbox/new-assignment", "assignment")
- 将严格的层次结构（树）变成有向图
 - 注意：用户不能为目录创建 link
- 不同的文件名可以指向同一个 inode 号

• UNLINK

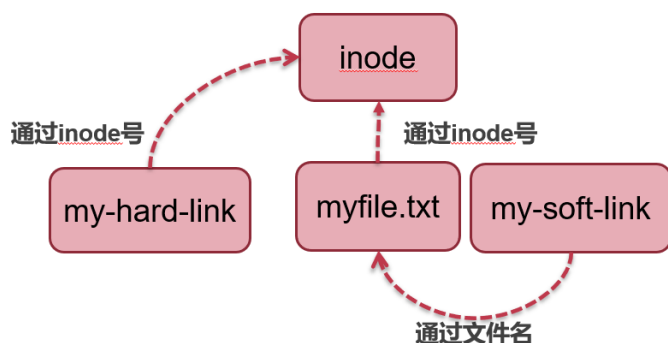
- 删掉从文件名到 inode 号的绑定关系
- 如果 UNLINK 最后一个绑定，则把 inode 和对应的 blocks 放到 free-list
 - 每个文件都需要一个 **reference counter**

• 引用计数器（Reference count）

- 一个 inode 可以绑定多个文件名
 - LINK 时 +1, UNLINK 时 -1
- 当 reference count 为 0 时，文件被删除
- 不允许出现环
 - 除了 '.' 和 '..'
 - 用来表明当前目录和上一层目录而不需要知道它们实际的名字

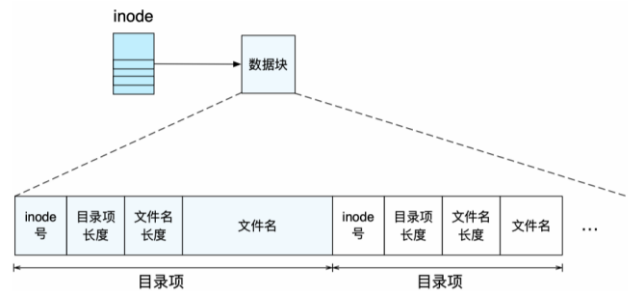


- /a/b 是一个目录
- a 的 refcnt 是 1
- a 的 inode 号是 25
- LINK ("a/b/c", "a")
- 造成一个环
- a 的 refcnt 是 2
- UNLINK ("a")
- a 的 refcnt 降为 1 (25号 inode 没有被删除)



软链接（符号链接）：一种特殊的 inode（快捷方式）

软链接可以无中生有



可以建立目录的软链接

文件的元数据（磁盘中）

• 拥有者/所在组 ID

- 拥有该 inode 的用户 ID 和 组 ID

• 权限的类型

- 拥有者、所在组、其他
- 读、写、执行

• 时间戳

- 最后一次访问 (如 READ 操作)
- 最后一次修改 (如 WRITE 操作)
- 最后一次 inode 更新 (如 LINK 操作)

文件元数据	说 明
mode	文件模式，其中包括文件类型和文件权限
nlink	指向此文件的链接个数
uid	文件所属用户的 ID
gid	文件所属用户组的 ID
size	文件的大小
atime	文件数据最近访问时间
ctime	文件元数据最近修改时间
mtime	文件数据最近修改时间

被打开文件的元数据（内存中）

• 整个系统维护了一个 file_table

- 记录了所有打开的文件的信息
- 包括：文件游标（file cursor）、引用数（ref_count）
- 父子进程间可以共享文件游标

• 每个进程维护了一个 fd_table

- 记录了该进程每个 fd 所对应文件在 file_table 中的索引

文件游标 Cursor

• 文件游标

- 记录了一个文件中下一次操作的位置
- 可以通过 SEEK 操作修改

• 情况 1: 共享游标

- 父进程将 fd 传递给子进程
 - UNIX 中，子进程会继承父进程所有已经打开的 fd
- 允许父子进程共享同一个文件

- **情况 2: 非共享游标**
 - 两个不同的进程打开同一个文件

write 和 close

- **write() 与 read() 类似**
 - 可能需要分配新的 block
 - 更新 inode 的 size 和 mtime
- **close()**
 - 释放 fd_table 中的相关项
 - 减小 file table 中相关项的 refcnt
 - 如果 file table 中相关项 refcnt 为 0, 则将其释放

删除一个打开的文件

- **进程 P1 打开了文件 A**
 - 运行 open, 在 file_table 和 fd_table 中都增加了一项
- **进程 P2 将文件 A 删除**
 - 删掉了指向文件 A 的最后一个目录项
 - 文件 A 的 inode 引用数变成了 0
- **文件 A 的 inode 不会被立即释放和删除**
 - 直到前一个进程调用 close 将其关闭
 - (在 Windows 上, 则通过"禁止删除打开的文件"实现类似效果)

崩溃一致性

方法-1: 同步元数据写+fsck

同步元数据写

- 每次元数据写入后, 运行 sync() 保证更新后的元数据入盘

若非正常重启, 则运行 fsck 检查磁盘, 具体步骤:

- **1. 检查 superblock**
 - 例: 保证文件系统大小大于已分配的磁盘块总和
 - 如果出错, 则尝试使用 superblock 的备份
- **2. 检查空闲的 block**
 - 扫描所有 inode 的所有包含的磁盘块
 - 用扫描结果来检验磁盘块的 bitmap
 - 对 inode bitmap 也用类似方法
- **3. 检查 inode 的状态**
 - 检查类型: 如普通文件、目录、符号链接等
 - 若类型错误, 则清除掉 inode 以及对应的 bitmap
- **4. 检查 inode 链接**
 - 扫描整个文件系统树, 核对文件链接的数量
 - 如果某个 inode 存在但不在任何一个目录, 则放到/lost+found
- **5. 检查重复磁盘块**
 - 如: 两个 inode 指向同一个磁盘块

- 如果一个 inode 明显有问题则删掉, 否则复制磁盘块一边给一个

6. 检查坏的磁盘块 ID

- 如: 指向超出磁盘空间的 ID

7. 检查目录

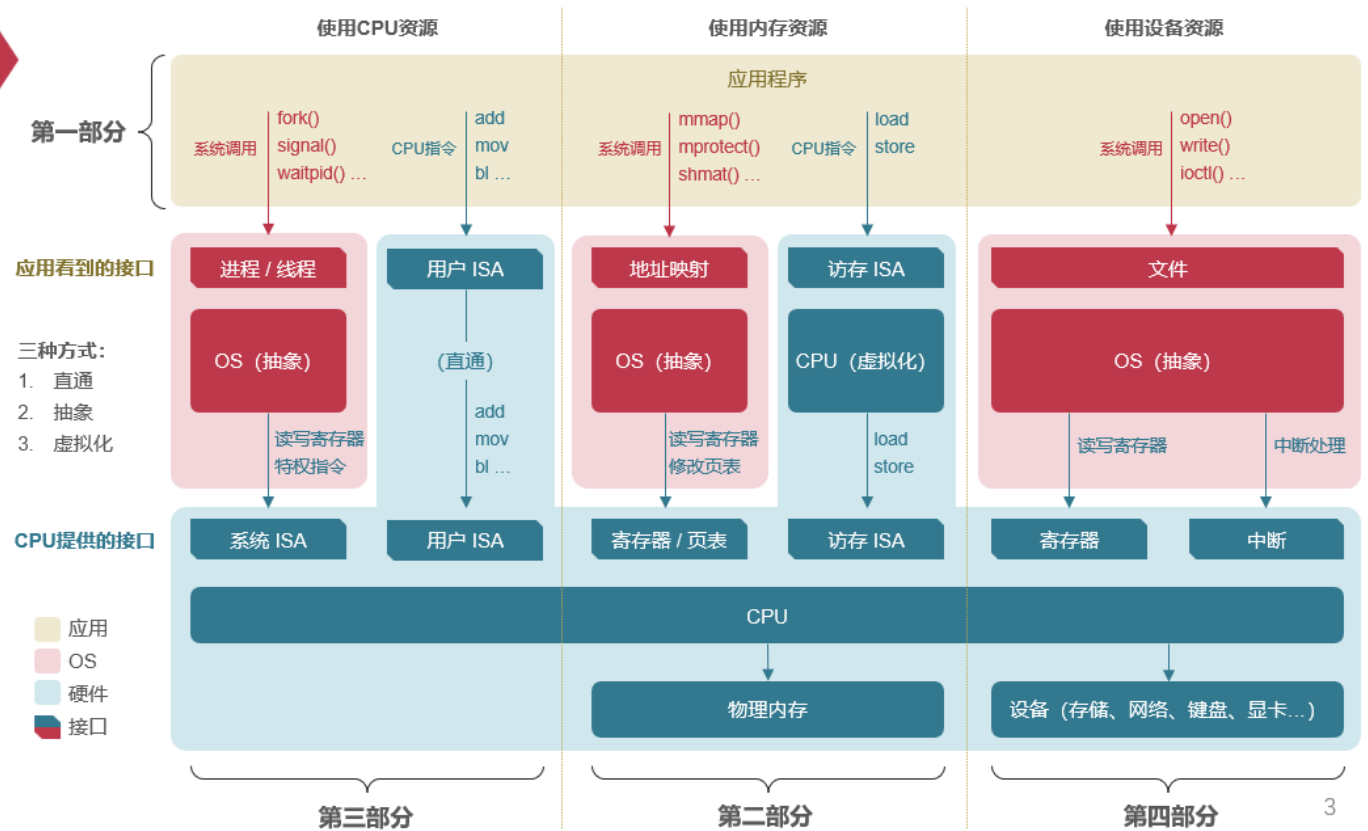
- 这是 fsck 对数据有更多语义的唯一的一种文件
- 保证 . 和 .. 是位于头部的目录项
- 保证目录中不会保证目录的链接数只能是 1 个
- 有相同的文件名

方法-2: 日志 (Journaling)

- **日志: 在磁盘上预留的专门空间**
- **在进行修改之前, 先将修改记录到日志中**
 - 如: 如何修改 block-bitmap、如何修改 data
- **所有要进行的修改都记录完毕后, 提交日志**
- **确定日志落盘后, 再修改数据和元数据**
- **修改完成后, 删除日志**
- **Data mode (即 full journaling)**
 - 数据和元数据都写入日志区域
- **Ordered mode (Default)**
 - 先写数据 (原本的文件位置), 再写元数据 (日志)
- **Writeback mode**
 - 仅仅将元数据写入日志
 - 数据依然写入原本的位置
 - 日志和数据之间没有顺序保证

Ordered Mode: 两次 Flush 保证顺序

- **启动后首先检查日志区域**
 - 若没有任何日志记录, 则无需恢复
- **扫描所有已经 COMMIT 的事务**
 - 若没有 COMMIT 的事务, 则无需恢复
 - 对已经 COMMIT 的事务, 将元数据从日志区写到原本位置
- **完成后清空日志区域**



程序可见状态

